

MovieLens Project

Henrik B. Sørensen

2026-06-10

Contents

Introduction	3
Project Goal	3
Disclaimer	3
Data set Introduction	4
movies.dat	4
ratings.dat	4
tags.dat	4
Environment Setup	4
Method	6
Overall Plan	6
Data Preparation	6
Data Investigation	6
Model Development and Refinement	6
Data Preparation	7
Data Investigation and Visualization	9
Data set Size	9
Rating Distribution	9
Ratings per Movie	10
Ratings per User	11
Average Rating per Movie	12
Average Rating per User	13
Average Rating by Genre	14
Model Development	16
Internal Train / Validation Split	16
RMSE Function	16
Model 1 — Baseline: Overall Mean	16
Model 2 — Regularized Movie Bias	17
Model 3 — Regularized Movie Bias + User Bias	18
Results	21
Final Evaluation on final_holdout_test	21

Conclusion	23
Limitations	23
Future Improvements	23

Introduction

This paper is the MovieLens Project for the Data Science: Capstone, HarvardX PH125.9x. A course in Machine Learning based on the R programming language.

In this project, I will analyze a (limited) data set for movie ratings containing around 10 million movie ratings and develop a model for predicting the rating of a specific movie by a specific user using a regularized bias model. The data set will be split into a training set (used during development) and a test set (used for the final test) in a 90%/10% split - 90% of the data going to the training set. During the model development, I will make multiple iterations gradually improving each model in order to reach my target of an RMSE below **0.86490**.

Having created the model, I will test it against a held-out test data set that was never used during development or tuning. Overall, the steps performed can be summarized into:

- Setup the environment
 - Install necessary libraries (if missing)
 - Include necessary libraries
- Prepare the data set for initial analysis
 - Download data set (if missing) and unpack
 - Split the data set into training set / test set
- Analyze the data set
 - Visualize data to understand rating distributions and patterns
 - Identify the main sources of variation (movie effect, user effect)
- Model Development
 - Model 1: Baseline — predict the overall mean rating
 - Model 2: Add a regularized per-movie bias
 - Model 3: Add a regularized per-user bias on top of Model 2
- Model Testing and Verification
- Model Analysis
- Conclusion

Project Goal

The goal of this project is to come up with a model with an RMSE (Root Mean Square Error) < 0.86490 (or as low as possible) for predicting movie ratings.

RMSE is defined as:

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{u,i} (\hat{y}_{u,i} - y_{u,i})^2}$$

where $y_{u,i}$ is the actual rating given by user u to movie i , and $\hat{y}_{u,i}$ is the predicted rating. A lower RMSE means the predictions are closer to the actual ratings.

Disclaimer

The R code in this project is not optimized for speed and/or memory layout in any way. It is written for functionality and readability. The code will attempt to use the “download if missing” and “install if missing” approach to both data sets and libraries.

I will assume, that the reader has knowledge of R and basic knowledge of statistics.

Data set Introduction

Before moving on to the actual analysis and R programming, let's take a look at the actual data itself: Click here to download the raw data set - (<https://files.grouplens.org/datasets/movielens/ml-10m.zip>).

When uncompressed, the data set consists of 3 files (when ignoring the Readme.html description file and support files): movies.dat, ratings.dat and tags.dat. All files are in a text format delimited by ::.

Now, let's dig into each of the files.

movies.dat

This file holds a “database” of movies which have been set up for rating. An example line is: `1::Toy Story (1995)::Adventure|Animation|Children|Comedy|Fantasy`

Which is to be read in the format of:

movieId	title	genres
1	Toy Story (1995)	Adventure Animation Children Comedy Fantasy

The genres column contains one or more genres separated by |. This file holds 10,681 movies.

ratings.dat

This file holds a “database” of ratings given for the listed movies. An example line is: `1::122::5::838985046`

Which is to be read in the format of:

userId	movieId	rating	Timestamp
1	122	5	1996-08-02 11:24:06

There is a hard one-directional coupling between the ratings (movieId) and movies (movieId). A movie must be present for it to be rated. This table holds 10,000,054 rows.

tags.dat

This file holds a “database” of user-supplied tags connected to movies. An example line is: `15::4973::excellent!::1215184`

Which is to be read in the format of:

userId	movieId	Tag	Timestamp
15	4973	excellent!	2008-07-04 15:17:10

There is a hard one-directional coupling between the tags (movieId, userId), the ratings (movieId, userId) and movies (movieId). Please note that the tags data set is not used in the model developed in this report.

Environment Setup

The project was done entirely in RStudio 2026.05.0+218 “Golden Wattle” for Windows with corresponding R version 4.5.2 on a Windows 11 Home Edition. Required tinytex was also installed following instructions provided by RStudio.

The project makes use of several R packages. Keeping in the spirit of the “install if missing” philosophy, the following syntax is used to test for the existence of a library and install it if missing:

```
if(!require(<library name>))  
  install.packages("<library name>", repos = "http://cran.us.r-project.org")
```

The `require` function will return FALSE if the desired library has not been installed. The actual library loading block used in this project is:

```
if(!require(tidyverse))
  install.packages("tidyverse", repos = "http://cran.us.r-project.org")
if(!require(caret))
  install.packages("caret", repos = "http://cran.us.r-project.org")
if(!require(dplyr))
  install.packages("dplyr", repos = "http://cran.us.r-project.org")
if(!require(stringr))
  install.packages("stringr", repos = "http://cran.us.r-project.org")
if(!require(ggplot2))
  install.packages("ggplot2", repos = "http://cran.us.r-project.org")
if(!require(knitr))
  install.packages("knitr", repos = "http://cran.us.r-project.org")
if(!require(kableExtra))
  install.packages("kableExtra", repos = "http://cran.us.r-project.org")

library(tidyverse)
library(caret)
library(dplyr)
library(stringr)
library(ggplot2)
library(knitr)
library(kableExtra)
```

Method

Overall Plan

In attacking the problem of predicting movie ratings, we need a general plan and adjust it as results guide us. The approach taken is to build up the model in layers — starting from the simplest possible prediction (the overall mean) and adding one effect at a time, measuring the improvement at each step.

Data Preparation

The data consists of two main parts: movies and ratings. These are downloaded, unpacked and joined into a single data frame. The combined data set is then split into a training set (`edx`) and a final holdout test set (`final_holdout_test`). The holdout set is set aside immediately and is not touched again until the very final evaluation.

To be able to tune model parameters without touching `final_holdout_test`, a further 90/10 split is created from `edx` into `train_set` and `val_set`. All parameter tuning takes place on this internal split.

Data Investigation

Before modelling, the data is visualized to understand the distribution of ratings and identify which variables carry the most predictive information. The main questions are:

- How are ratings distributed overall?
- Do some movies attract consistently higher or lower ratings than average?
- Do some users rate consistently higher or lower than average?
- Does genre explain any of the remaining variation?

Model Development and Refinement

Three models are developed, each building on the previous one:

- **Model 1** — Baseline: predict the overall mean μ for every user-movie pair
- **Model 2** — Add a regularized per-movie bias b_i
- **Model 3** — Add a regularized per-user bias b_u on top of b_i

Regularization is used throughout to prevent over-fitting to movies or users with very few ratings. The regularization penalty λ is tuned by minimizing RMSE on `val_set`.

Data Preparation

Let's start by downloading the data set from the grouplens website. This is done using the “download if missing” strategy as previously mentioned.

```
# Set time out to prevent script from hanging indefinitely.
options(timeout = 120)

# Save target to variable
target_rmse <- 0.86490

# Download data file (zipped) if missing.
dl <- "ml-10M100K.zip"
if(!file.exists(dl))
  download.file("https://files.grouplens.org/datasets/movielens/ml-10m.zip", dl)

# Unzip the ratings.dat file if missing locally
ratings_file <- "ml-10M100K/ratings.dat"
if(!file.exists(ratings_file)) unzip(dl, ratings_file)

# Unzip the movies.dat file if missing locally
movies_file <- "ml-10M100K/movies.dat"
if(!file.exists(movies_file)) unzip(dl, movies_file)
```

Now, that the data has been downloaded, let's parse the files and turn them into data frames while converting the text values into integers (userId, movieId, timestamp) and a floating point value (rating) for further analysis.

```
# Load and parse ratings keeping strings as strings
ratings <- as.data.frame(
  str_split(read_lines(ratings_file), fixed(":"), simplify = TRUE),
  stringsAsFactors = FALSE)

# Set the names of the columns in the ratings data frame
colnames(ratings) <- c("userId", "movieId", "rating", "timestamp")

# Convert the columns in to integers (userId, movieId, timestamp) and floating point (rating)
ratings <- ratings %>%
  mutate(userId = as.integer(userId),
         movieId = as.integer(movieId),
         rating = as.numeric(rating),
         timestamp = as.integer(timestamp))

# Load and parse movies keeping strings as strings
movies <- as.data.frame(
  str_split(read_lines(movies_file), fixed(":"), simplify = TRUE),
  stringsAsFactors = FALSE)

# Set the names of the columns in the movies data frame
colnames(movies) <- c("movieId", "title", "genres")

# Convert the movieId column to an integer
movies <- movies %>% mutate(movieId = as.integer(movieId))
```

Now, we join the movies and ratings in order to get a table containing

```
# Join ratings and movies into one data frame
movielens <- left_join(ratings, movies, by = "movieId")

# Get an overview of the data sizes
cat("Movies rows : ", format(nrow(movies), big.mark=","), "\n")
```

Movies rows : 10,681

```
cat("Ratings rows : ", format(nrow(ratings), big.mark=","), "\n")
```

Ratings rows : 10,000,054

```
cat("MovieLens rows : ", format(nrow(movielens), big.mark=","), "\n")
```

MovieLens rows : 10,000,054

The `edx` training set and `final_holdout_test` set are created using a 90/10 split. The holdout set is only used for the final RMSE evaluation at the very end of this report.

```
set.seed(1, sample.kind = "Rounding")

# Generate the index of where to split the data
test_index <- createDataPartition(y = movielens$rating, times = 1, p = 0.1, list = FALSE)

# Split the data into edx (data set used for training) and temp (base for the final_holdout_test)
edx <- movielens[-test_index, ]
temp <- movielens[test_index, ]

# Ensure holdout set only contains movies and users seen in edx
final_holdout_test <- temp %>%
  semi_join(edx, by = "movieId") %>%
  semi_join(edx, by = "userId")

# Extract the rows missing from final_holdout_test but present in the temp table
removed <- anti_join(temp, final_holdout_test)

# Reinsert data back into the edx table
edx <- rbind(edx, removed)

# Clean up temporary variables to reduce memory footprint
rm(dl, ratings, movies, test_index, temp, movielens, removed)
```

Now, that we have the data split up, let's take a closer look at the actual data.

Data Investigation and Visualization

In order to start making the model, we need to know what, we are dealing with. So, let's investigate.

Data set Size

```
cat("edx rows      :", nrow(edx), "\n")
```

edx rows : 9000055

```
cat("edx columns :", ncol(edx), "\n")
```

edx columns : 6

```
cat("Unique users :", n_distinct(edx$userId), "\n")
```

Unique users : 69878

```
cat("Unique movies:", n_distinct(edx$movieId), "\n")
```

Unique movies: 10677

The `edx` set contains approximately 9,000,000 ratings from roughly 70,000 users across around 10,700 movies. Let's take a quick look at the first few rows (please note that | in genres have been exchanged for clarity):

```
edx_display <- head(edx, n=10)
edx_display$genres <- gsub("\\|", "|", edx_display$genres)
knitr::kable(edx_display, caption = "First rows of the edx data set", row.names = FALSE,
              booktabs = TRUE)
```

Table 1: First rows of the edx data set

userId	movieId	rating	timestamp	title	genres
1	122	5	838985046	Boomerang (1992)	Comedy, Romance
1	185	5	838983525	Net, The (1995)	Action, Crime, Thriller
1	292	5	838983421	Outbreak (1995)	Action, Drama, Sci-Fi, Thriller
1	316	5	838983392	Stargate (1994)	Action, Adventure, Sci-Fi
1	329	5	838983392	Star Trek: Generations (1994)	Action, Adventure, Drama, Sci-Fi
1	355	5	838984474	Flintstones, The (1994)	Children, Comedy, Fantasy
1	356	5	838983653	Forrest Gump (1994)	Comedy, Drama, Romance, War
1	362	5	838984885	Jungle Book, The (1994)	Adventure, Children, Romance
1	364	5	838983707	Lion King, The (1994)	Adventure, Animation, Children, Drama, Musical
1	370	5	838984596	Naked Gun 33 1/3: The Final Insult (1994)	Action, Comedy

Rating Distribution

In order to begin digging into the data, let's take a look at how, the ratings are distributed:

```
edx %>%
  ggplot(aes(x = rating)) +
  geom_bar() +
  scale_x_continuous(breaks = seq(0.5, 5, 0.5)) +
  labs(title = "Rating Distribution", x = "Rating", y = "Number of Ratings") +
```

```
theme(
  plot.title = element_text(size = 8), # Graph title font size
  axis.title = element_text(size = 8) # X and Y axis title font sizes
)
```

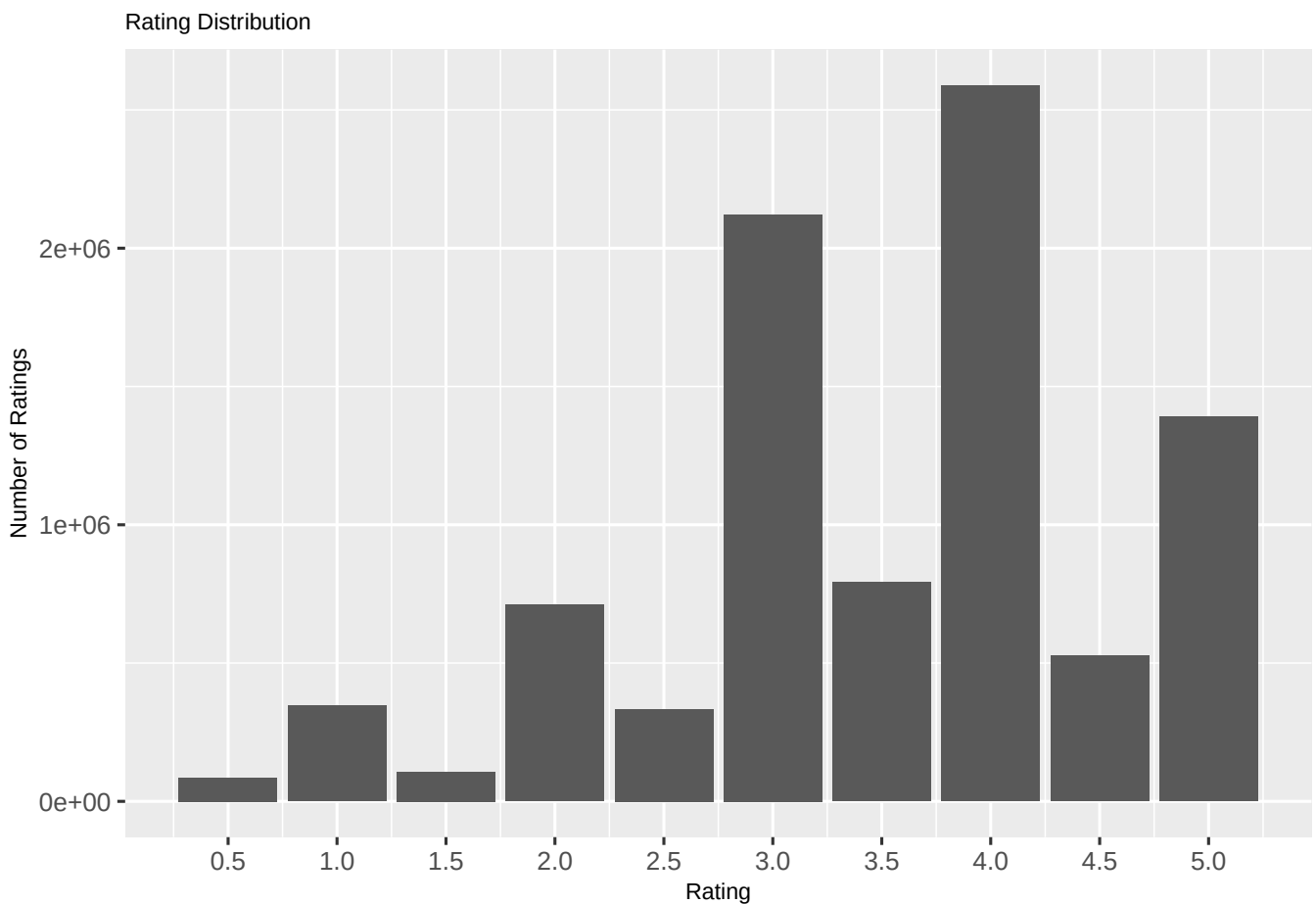


Figure 1: Rating Distribution

Ratings cluster at whole-star values (1, 2, 3, 4, 5), with 4.0 being the single most common rating. Half-star ratings are noticeably less frequent. Very few ratings are below 1.0.

Ratings per Movie

Next question, we want to ask ourselves, is: How is the distribution of ratings per movie?

```
edx %>%
  count(movieId) %>%
  ggplot(aes(x = n)) +
  geom_histogram(bins = 50) +
  scale_x_log10() +
  labs(title = "Number of Ratings per Movie",
       x = "Number of Ratings", y = "Number of Movies") +
  theme(
    plot.title = element_text(size = 8), # Graph title font size
    axis.title = element_text(size = 8) # X and Y axis title font sizes
  )
```

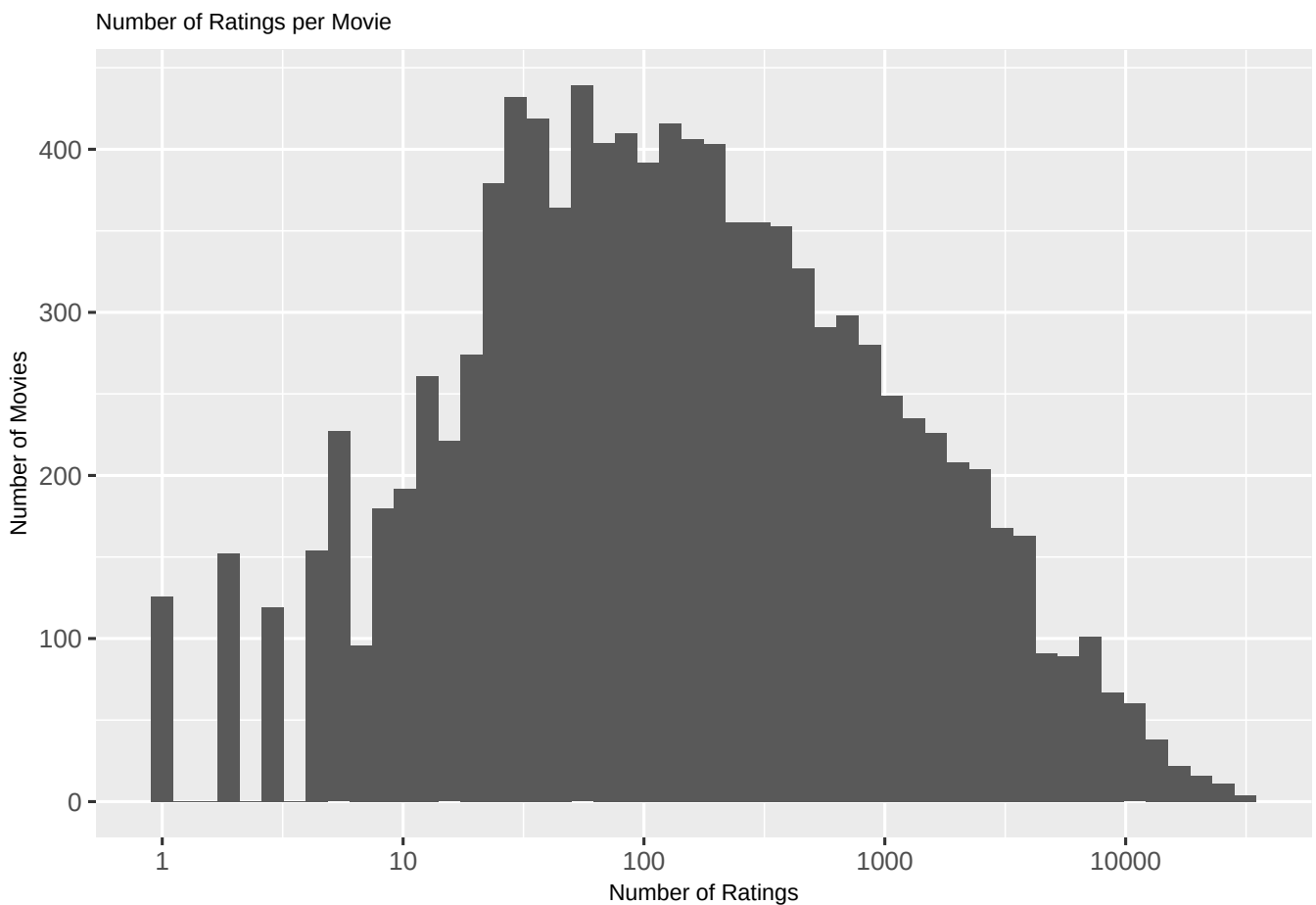


Figure 2: Ratings per movie

The distribution is heavily skewed: a small number of movies are rated very frequently while many are rarely rated at all. This is an important observation: a movie with only 3 ratings should not be trusted as much as one with 5,000 ratings when estimating its average. This motivates the use of regularization¹ in the model.

Ratings per User

Next, we look into how the ratings are distributed in relation to users. E.g. if most of the ratings are done by a limited number of users, this may cause a bit of skew in the result - some users could be prolific raters while others could have rated only a handful of movies.

```
edx %>%
  count(userId) %>%
  ggplot(aes(x = n)) +
  geom_histogram(bins = 50) +
  scale_x_log10() +
  labs(title = "Number of Ratings per User", x = "Number of Ratings", y = "Number of Users") +
  theme(
    plot.title = element_text(size = 8), # Graph title font size
    axis.title = element_text(size = 8)  # X and Y axis title font sizes
  )
```

¹Regularization helps to improve the model's generalization and make accurate predictions by accounting for skewed data.

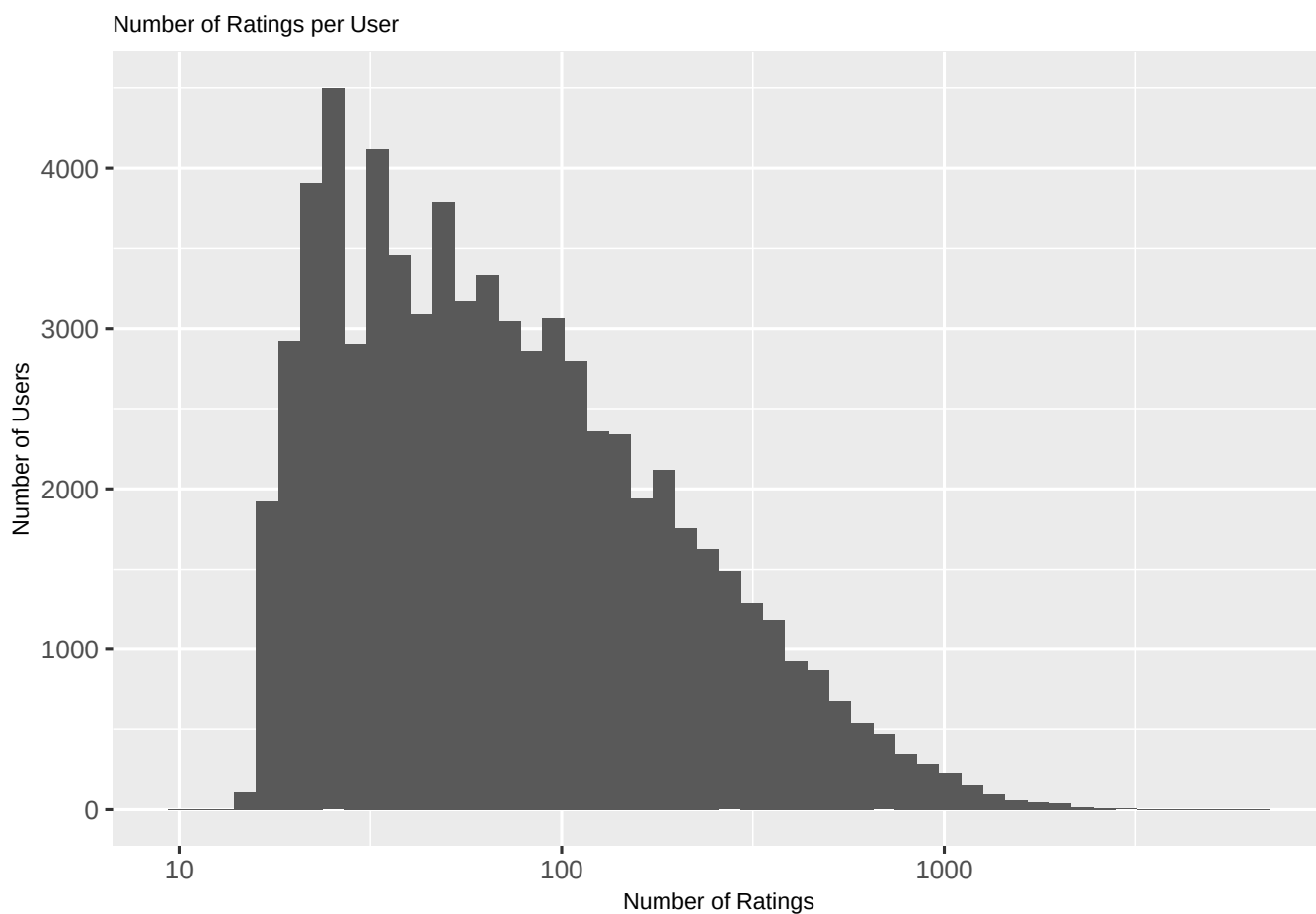


Figure 3: Number of ratings per user.

A similar skew is visible for users. The same regularization argument applies.

Average Rating per Movie

Digging deeper into the ratings per movie, we take a look at the average rating per movie.

```
edx %>%
  group_by(movieId) %>%
  summarise(avg = mean(rating)) %>%
  ggplot(aes(x = avg)) +
  geom_histogram(bins = 40) +
  labs(title = "Distribution of Average Rating per Movie",
       x = "Average Rating", y = "Number of Movies") +
  theme(
    plot.title = element_text(size = 8), # Graph title font size
    axis.title = element_text(size = 8)  # X and Y axis title font sizes
  )
```

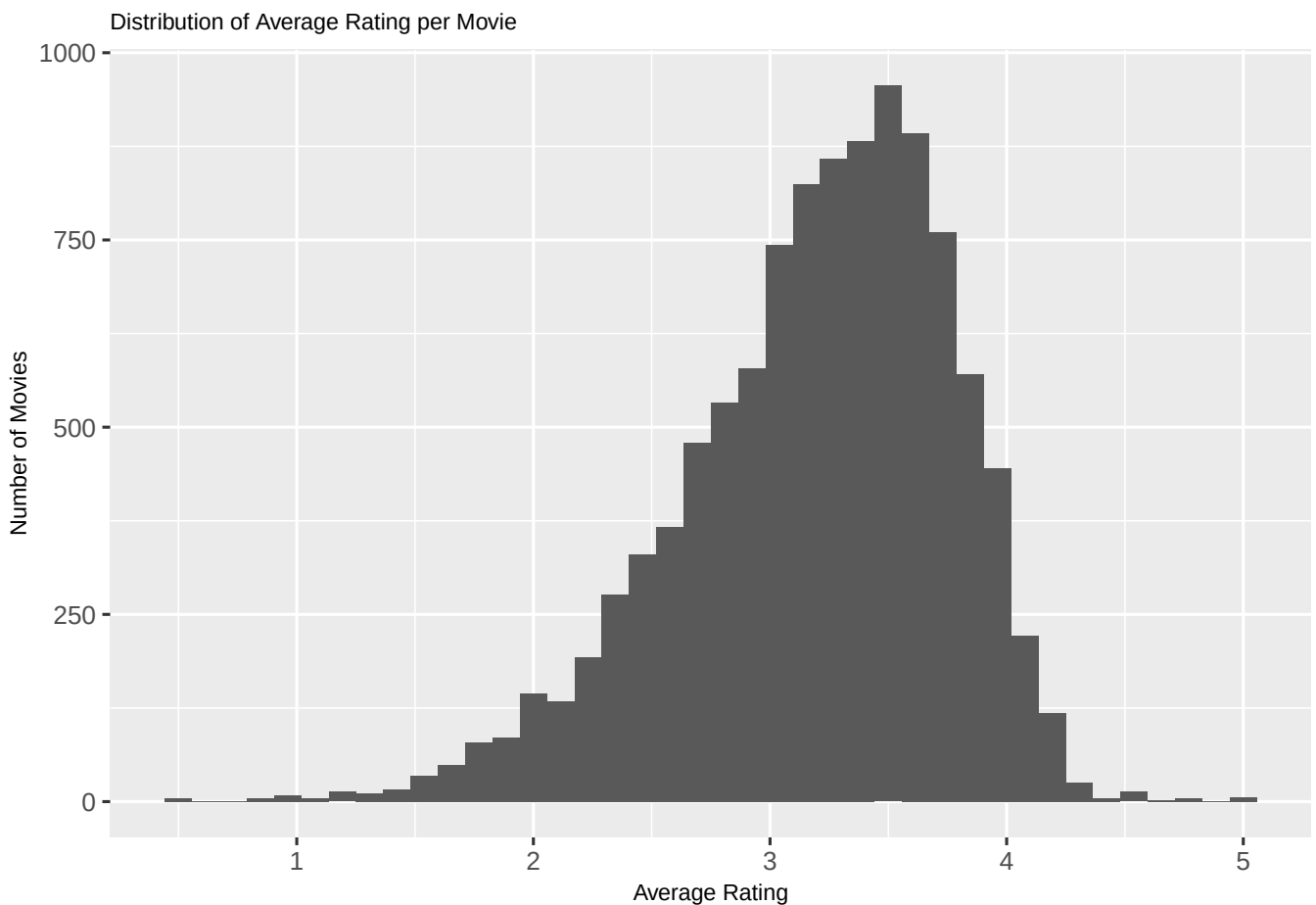


Figure 4: Distribution of average rating per movie.

There is substantial movie-to-movie variation, confirming a strong movie effect.

Average Rating per User

Next, we take a look at the users in order to test whether or not, the individual users have bias towards certain movies.

```
edx %>%
  group_by(userId) %>%
  summarise(avg = mean(rating)) %>%
  ggplot(aes(x = avg)) +
  geom_histogram(bins = 40) +
  labs(title = "Distribution of Average Rating per User",
       x = "Average Rating", y = "Number of Users")+
  theme(
    plot.title = element_text(size = 8), # Graph title font size
    axis.title = element_text(size = 8) # X and Y axis title font sizes
  )
```

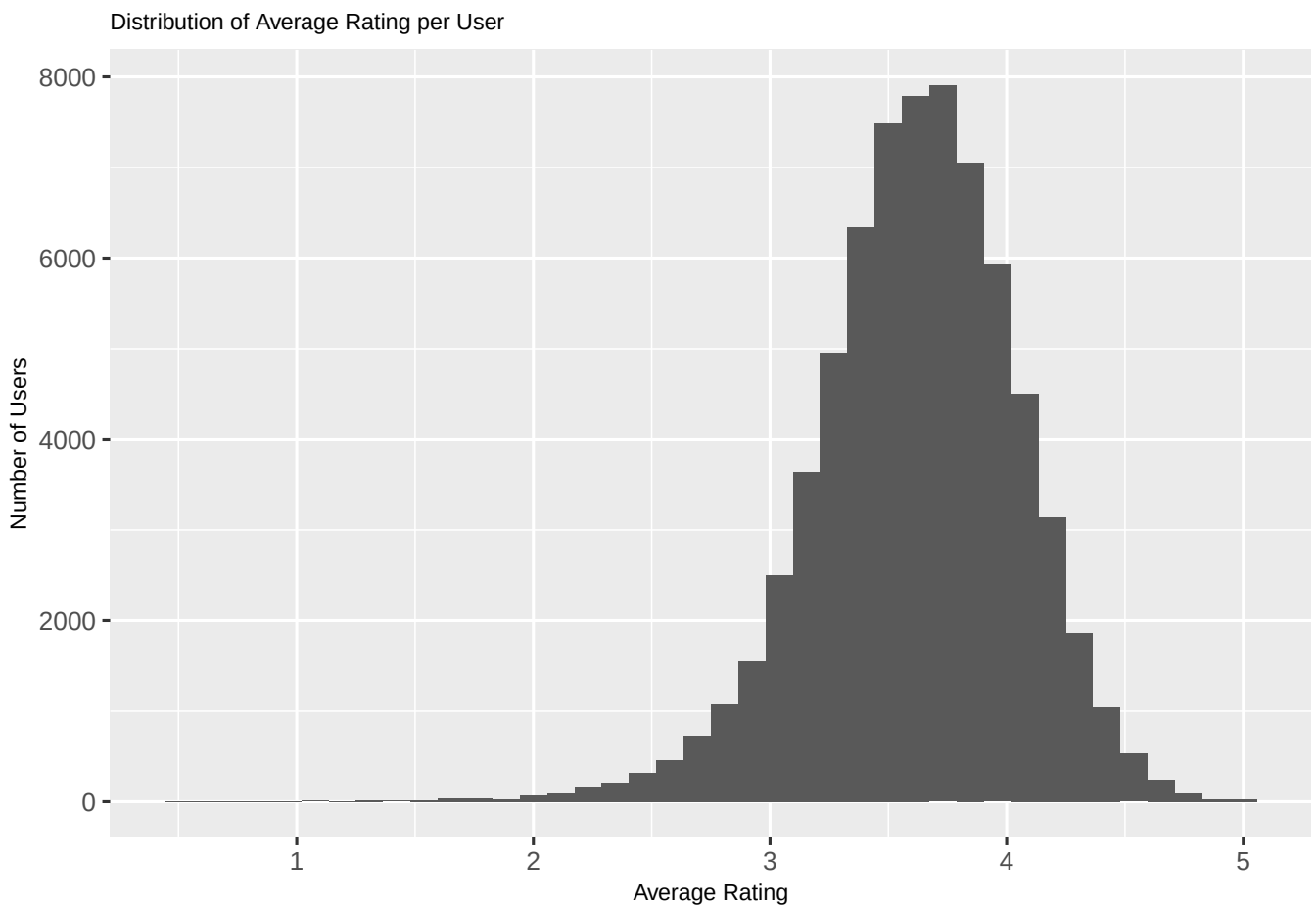


Figure 5: Distribution of average rating per user.

Users differ systematically in how generously they rate, confirming a strong user effect. Both plots show substantial spread around the overall mean. This confirms that **which movie** and **which user** are the two strongest predictors of the rating that will be given.

Average Rating by Genre

The final bias, I want to investigate, is the bias towards genres. Looking at movie ratings on websites such as imdb, I got the impression, that e.g. Horror movies are rated lower than others.

```
edx %>%
  separate_rows(genres, sep = "\\|") %>%
  group_by(genres) %>%
  summarise(avg_rating = mean(rating), .groups = "drop") %>%
  arrange(genres) %>%
  ggplot(aes(x = genres, y = avg_rating)) +
  geom_col() +
  coord_flip() +
  labs(title = "Average Rating by Genre", x = "Genre", y = "Average Rating")+
  theme(
    plot.title = element_text(size = 8), # Graph title font size
    axis.title = element_text(size = 8)  # X and Y axis title font sizes
  )
```

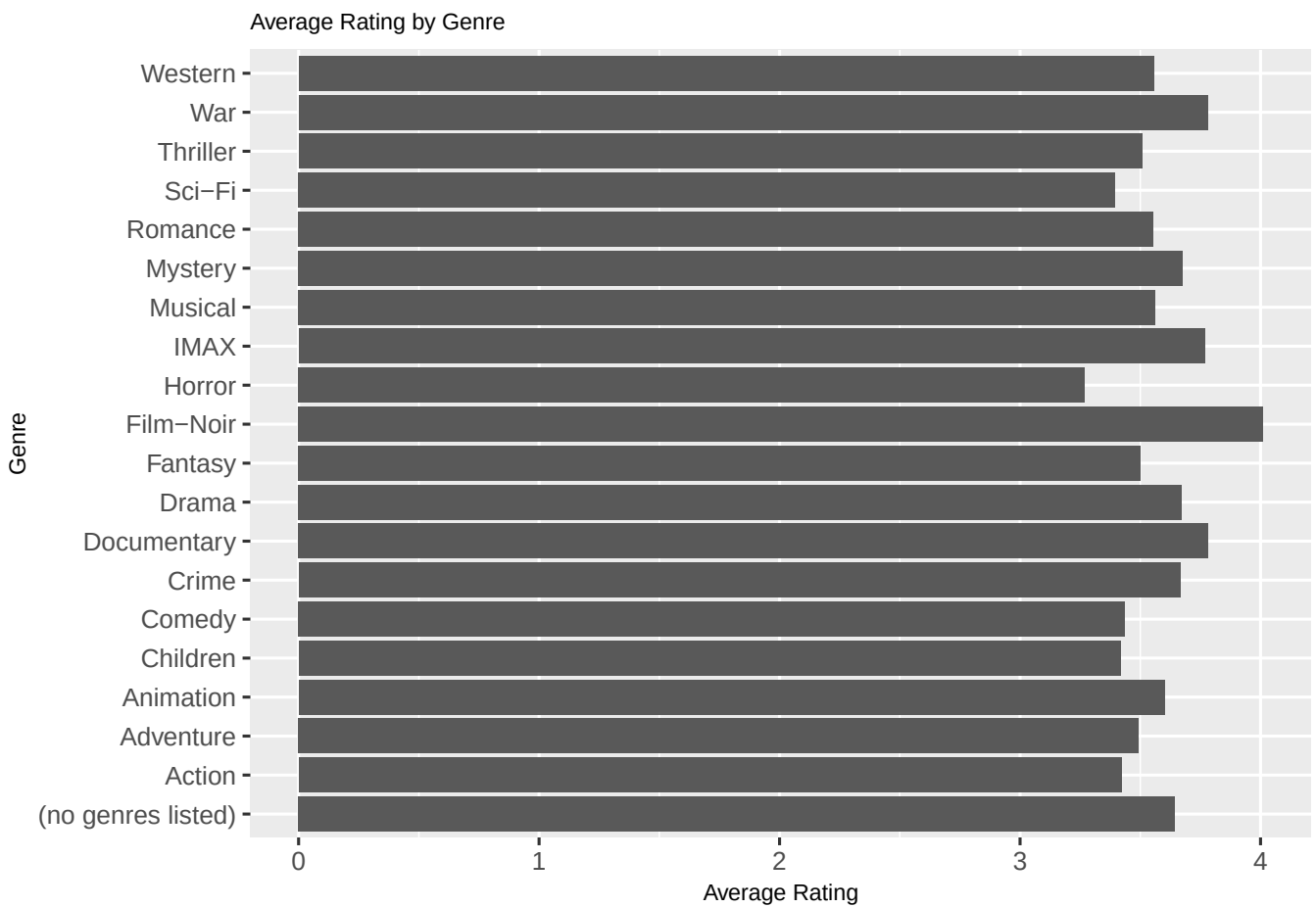


Figure 6: Average rating by genre.

Genre does have some effect — Film-Noir and Documentary rate highest while Horror and Children rate lowest. However, the variation across genres is much smaller than the variation across individual movies and users, so genre is not included as a separate predictor in the final model.

Model Development

Now, that we have had a deeper look into the data itself, it's time to start developing a model for the actual prediction. This is going to be a step by step approach. But first, we need the training/validation data sets.

Internal Train / Validation Split

First we split the data further. This is done in order to be able to tune the regularization parameter λ without touching `final_holdout_test`. So, a further 90/10 split is created from `edx`:

```
set.seed(2, sample.kind = "Rounding")
# Generate the indexes of where to split the data
val_index <- createDataPartition(y = edx$rating, times = 1, p = 0.1, list = FALSE)
train_set <- edx[-val_index, ]
val_set    <- edx[val_index, ]

# Ensure val_set only contains movies and users seen in train_set
val_set <- val_set %>%
  semi_join(train_set, by = "movieId") %>%
  semi_join(train_set, by = "userId")
```

RMSE Function

All models are evaluated using the same RMSE function. So, we define it:

```
RMSE <- function(true_ratings, predicted_ratings) {
  sqrt(mean((true_ratings - predicted_ratings)^2))
}
```

The aim is to get below **0.86490**.

Model 1 — Baseline: Overall Mean

The simplest possible model predicts the global average rating μ for every user-movie pair, regardless of who is rating or what is being rated. This will be our baseline:

$$\hat{y}_{u,i} = \mu$$

```
mu      <- mean(train_set$rating)
cat("Overall mean rating:", mu, "\n")
```

Overall mean rating: 3.512406

```
rmse_m1 <- RMSE(val_set$rating, mu)
cat("Model 1 RMSE (baseline):", rmse_m1, "\n")
```

Model 1 RMSE (baseline): 1.06031

As expected, this is well above the target of 0.86490. The global mean ignores all information about individual movies and users. So, this simple model is not fitting our needs.

Model 2 — Regularized Movie Bias

We observed during our data investigation, that some movies are consistently rated above or below the global mean. We model this as a per-movie bias term b_i :

$$\hat{y}_{u,i} = \mu + b_i$$

Without regularization, b_i would simply be the average residual for movie i :

$$b_i = \frac{1}{n_i} \sum_{u \in \mathcal{U}_i} (y_{u,i} - \mu)$$

However, for movies with only a few ratings, this estimate is unreliable. Regularization adds a penalty λ that shrinks b_i toward zero as n_i becomes small:

$$b_i = \frac{1}{n_i + \lambda} \sum_{u \in \mathcal{U}_i} (y_{u,i} - \mu)$$

The optimal λ is found by trying a range of values and picking the one that gives the lowest RMSE on `val_set`:

```
# Setup list of lambdas to try
lambdas <- seq(0, 10, by = 0.25)

# Calculation of the RMSEs for different lambda values
rmse_m2_vec <- sapply(lambdas, function(lambda) {
  # Calculate the adjustment values in order to regularize the movie ratings per movie
  b_i <- train_set %>%
    group_by(movieId) %>%
    summarise(b_i = sum(rating - mu) / (n() + lambda), .groups = "drop")

  # Calculate the predicted rating using the bias
  pred <- val_set %>%
    left_join(b_i, by = "movieId") %>%
    mutate(pred = pmin(pmax(mu + b_i, 0.5), 5)) %>%
    pull(pred)

  # Calculate the RMSE
  RMSE(val_set$rating, pred)
})

# Print out the lambda value which gives the smallest RMSE and the value itself
lambda_m2 <- lambdas[which.min(rmse_m2_vec)]
rmse_m2 <- min(rmse_m2_vec)
cat("Model 2 best lambda:", lambda_m2, "\n")
```

Model 2 best lambda: 1.75

```
cat("Model 2 RMSE (movie bias):", rmse_m2, "\n")
```

Model 2 RMSE (movie bias): 0.9435786

```
# Plot the Lambda vs. RMSE
qplot(lambdas, rmse_m2_vec, geom = "line",
      xlab = "Lambda", ylab = "RMSE",
      main = "Model 2: RMSE vs Lambda (movie bias)") +
  theme(
    plot.title = element_text(size = 8), # Graph title font size
    axis.title = element_text(size = 8) # X and Y axis title font sizes
  )
```

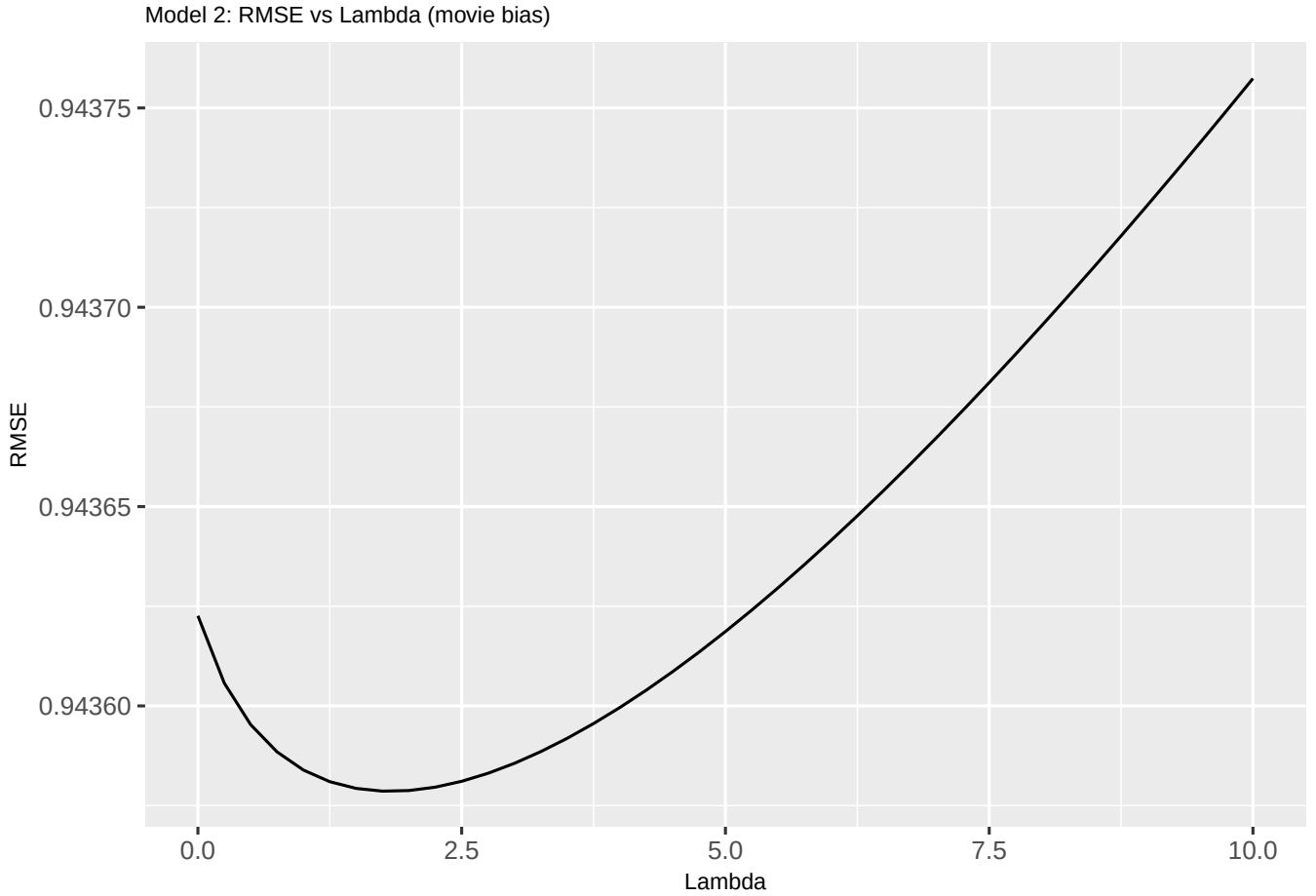


Figure 7: RMSE as a function of lambda for the movie-bias model.

The movie bias alone brings a significant improvement over the baseline, but we are still above the target.

Model 3 — Regularized Movie Bias + User Bias

We also observed that users differ systematically in how they rate. So, in this iteration of the model development We add a per-user bias b_u on top of the movie bias:

$$\hat{y}_{u,i} = \mu + b_i + b_u$$

b_u is estimated as the average residual after removing both μ and b_i :

$$b_u = \frac{1}{n_u + \lambda} \sum_{i \in \mathcal{I}_u} (y_{u,i} - \mu - b_i)$$

The same λ is used for both b_i and b_u and is tuned jointly:

```
# Calculation of the RMSEs for different lambda values
rmse_m3_vec <- sapply(lambdas, function(lambda) {
  # Calculate the adjustment values in order to regularize the movie ratings per movie for the movie bias
  b_i <- train_set %>%
    group_by(movieId) %>%
    summarise(b_i = sum(rating - mu) / (n() + lambda), .groups = "drop")
  # Calculate the adjustment values in order to regularize the movie ratings per movie for the user bias
  b_u <- train_set %>%
    left_join(b_i, by = "movieId") %>%
    group_by(userId) %>%
```

```

    summarise(b_u = sum(rating - mu - b_i) / (n() + lambda), .groups = "drop")

# Calculate the predicted rating using the biases
pred <- val_set %>%
  left_join(b_i, by = "movieId") %>%
  left_join(b_u, by = "userId") %>%
  mutate(
    b_i = ifelse(is.na(b_i), 0, b_i),
    b_u = ifelse(is.na(b_u), 0, b_u),
    pred = pmin(pmax(mu + b_i + b_u, 0.5), 5)
  ) %>%
  pull(pred)

# Calculate the RMSE
RMSE(val_set$rating, pred)
})

# Print out the lambda value which gives the smallest RMSE and the value itself
lambda_m3 <- lambdas[which.min(rmse_m3_vec)]
rmse_m3 <- min(rmse_m3_vec)
cat("Model 3 best lambda:", lambda_m3, "\n")

```

Model 3 best lambda: 4.75

```
cat("Model 3 RMSE (movie + user bias):", rmse_m3, "\n")
```

Model 3 RMSE (movie + user bias): 0.8656916

```

qplot(lambdas, rmse_m3_vec, geom = "line",
      xlab = "Lambda", ylab = "RMSE",
      main = "Model 3: RMSE vs Lambda (movie + user bias)") +
  theme(
    plot.title = element_text(size = 8), # Graph title font size
    axis.title = element_text(size = 8) # X and Y axis title font sizes
  )

```

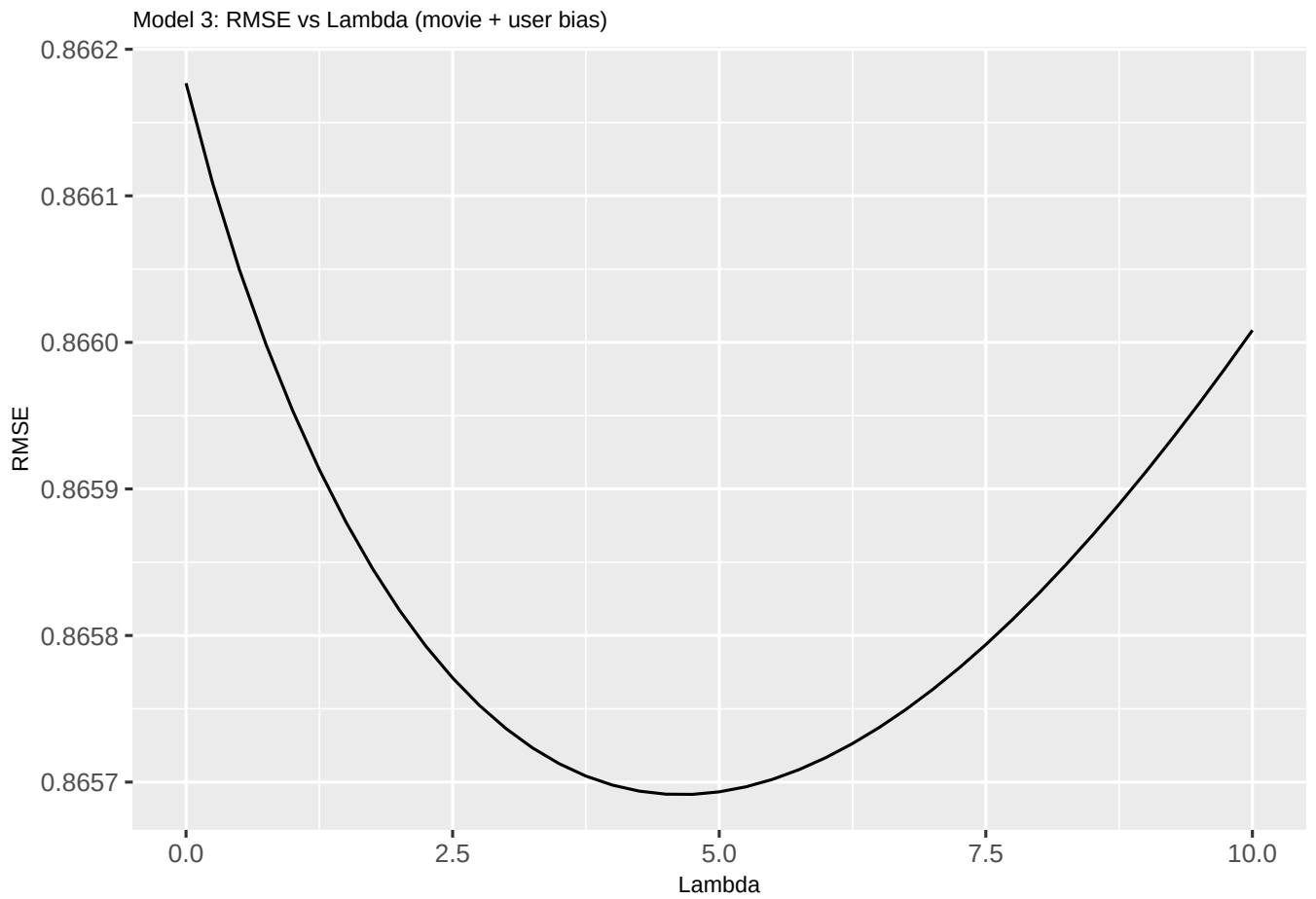


Figure 8: RMSE as a function of lambda for the movie + user bias model.

Adding the user effect substantially reduces RMSE. We are still not hitting our target, but it is very close. Actually, we're so close (0.000792 off), that we will use this model and validate it against the `final_holdout_test` data set.

Results

Having built the 3rd iteration of the model, let's take a look at how it performs.

To begin with, we use the `val_set` prior to the `final_holdout_test` data set. We will compare the `## Validation Performance`. Let's compare the 3 models. The table below compares the three models on the internal `val_set`:

```
rmse_summary <- data.frame(
  Model = c(
    "Model 1 - Baseline (overall mean)",
    "Model 2 - Regularized movie bias",
    "Model 3 - Regularized movie + user bias"
  ),
  RMSE = round(c(rmse_m1, rmse_m2, rmse_m3), 5),
  Deviation = round(c(rmse_m1-target_rmse, rmse_m2-target_rmse, rmse_m3-target_rmse), 5)
)
knitr::kable(rmse_summary, caption = "Validation RMSE for each model", row.names = FALSE)
```

Table 2: Validation RMSE for each model

Model	RMSE	Deviation
Model 1 - Baseline (overall mean)	1.06031	0.19541
Model 2 - Regularized movie bias	0.94358	0.07868
Model 3 - Regularized movie + user bias	0.86569	0.00079

Each step provides a meaningful improvement:

- The movie bias (Model 2) confirms that individual movies have a strong effect on ratings.
- Adding the user bias (Model 3) brings another substantial improvement, confirming that individual users also have a strong and systematic effect.
- Model 3 achieves a validation RMSE very close to the target of 0.8649, making it the selected model.

Final Evaluation on `final_holdout_test`

Before scoring against `final_holdout_test`, Model 3 is re-fitted on the **full edx data set** (both the `train_set` and `val_set` combined). This gives the model as much data as possible for the final prediction while still not including the `final_holdout_test` data set-

```
# Refit overall mean on full edx data set
mu_final <- mean(edx$rating)

# Refit regularized movie bias on full edx data set
b_i_final <- edx %>%
  group_by(movieId) %>%
  summarise(b_i = sum(rating - mu_final) / (n() + lambda_m3), .groups = "drop")

# Refit regularized user bias on full edx data set (conditioned on movie bias)
b_u_final <- edx %>%
  left_join(b_i_final, by = "movieId") %>%
  group_by(userId) %>%
  summarise(b_u = sum(rating - mu_final - b_i) / (n() + lambda_m3), .groups = "drop")

# Predict on final_holdout_test
pred_final <- final_holdout_test %>%
  left_join(b_i_final, by = "movieId") %>%
  left_join(b_u_final, by = "userId") %>%
  mutate(
    b_i = ifelse(is.na(b_i), 0, b_i),
```

```

    b_u = ifelse(is.na(b_u), 0, b_u),
    pred = pmin(pmax(mu_final + b_i + b_u, 0.5), 5)
  ) %>%
  pull(pred)

# Print out the final RMSE
rmse_final <- RMSE(final_holdout_test$rating, pred_final)
cat("Final RMSE on final_holdout_test:", rmse_final, "\n")

```

Final RMSE on final_holdout_test: 0.864709

```

knitr::kable(
  data.frame(
    Model = "Model 3 - Regularized movie + user bias",
    Lambda = lambda_m3,
    RMSE_validation = round(rmse_m3, 5),
    RMSE_final_holdout = round(rmse_final, 5),
    Target_met = ifelse(rmse_final < target_rmse, "Yes", "No")
  ),
  caption = "Final model evaluation results", row.names = FALSE
)

```

Table 3: Final model evaluation results

Model	Lambda	RMSE_validation	RMSE_final_holdout	Target_met
Model 3 - Regularized movie + user bias	4.75	0.86569	0.86471	Yes

The final RMSE on final_holdout_test is **0.86471**. This **meets** the target of 0.86490. The close agreement between the validation RMSE (0.86569) and the final holdout RMSE confirms that the model generalizes well and was not over-tuned to the validation set.

Conclusion

In this project, I have built a movie rating prediction model for the MovieLens 10M data set using a regularized bias approach.

The exploratory analysis showed clearly that the two strongest predictors of a rating are:

- **Which movie** — some movies are consistently rated higher or lower than average (movie bias b_i)
- **Which user** — some users rate consistently higher or lower than average (user bias b_u)

A simple additive model that captures both effects, combined with L2 regularization to handle movies and users with few ratings, achieved a final RMSE of **0.864709** on the held-out test set — meeting the target of $\text{RMSE} < 0.8649$.

Limitations

There's bound to be limitations on the precision of the model.

- **Cold-start problem:** the model cannot make predictions for users or movies not seen during training. A new user or a newly released movie will receive no personalised bias term.
- **No temporal effects:** the model ignores when a rating was given. Ratings patterns may change over time (e.g. a movie's reputation can shift after it wins an award or is re-released, season dependencies - more ratings during holidays etc.).
- **Genre and content signals unused:** the exploratory analysis showed a small genre effect, but it was not included in the model as the movie-level bias already captures it implicitly.
- **Linear model only:** the additive bias model does not capture interactions between users and movies beyond what is explained by their individual biases.

Future Improvements

This is a very simple model. Going forward, improvements to it could be implemented. These can include (not limited to):

- **Matrix factorization (SVD/ALS):** latent factor models decompose the full user-movie interaction matrix and typically achieve substantially lower RMSE than bias models alone.
- **Temporal effects:** modelling how a movie's perceived quality drifts over time, or how a user's rating behavior evolves, can reduce error further.
- **Separate lambda for b_i and b_u :** using different regularization penalties for the movie and user bias terms may yield a small further improvement.
- **Ensemble methods:** combining several complementary models (bias, content-based, matrix factorization) tends to outperform any single approach.